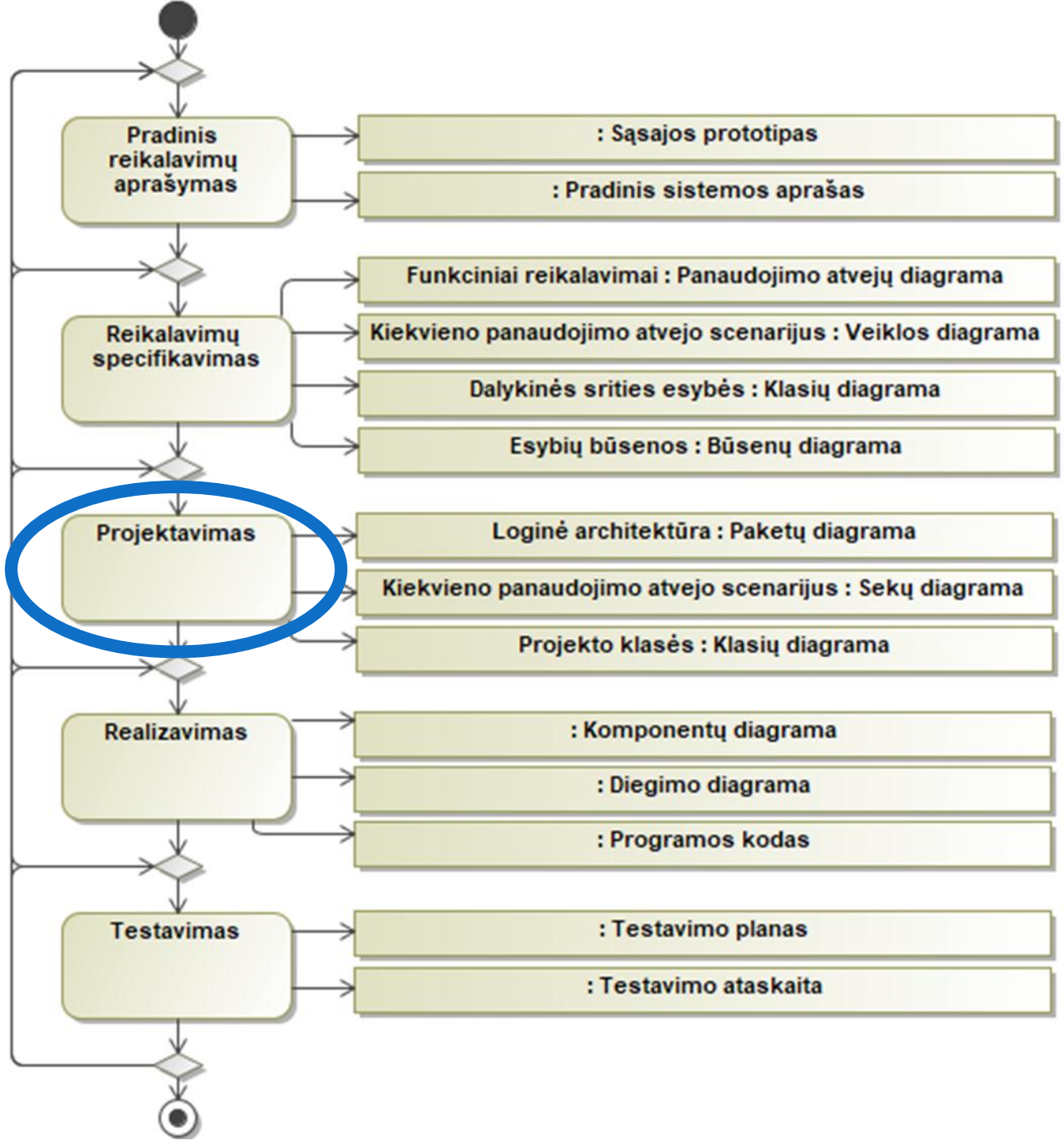
Naudojamas PĮ kūrimo procesas



ktu

Sekų diagrama

SEQUENCE DIAGRAM

Saveikų (*Interaction*) diagramos

ktu

Sąveika – elgsena, sudaryta iš aibės pranešimų, kuriais keičiasi objektai, atliekantys tam tikrus vaidmenis siekdami tam tikro tikslo.

Sąveikų diagramų tipai:

- **Sekų diagramos** (*Sequence diagrams*) – dažniausiai naudojamos
- Bendradarbiavimo diagramos (*Communication diagrams*)
- Laiko diagramos (*Timing diagrams*)
- Sąveikų apžvalgos diagramos (*Interaction overview diagrams*)

Pagrindiniai sekų diagramos elementai

ktu

Gyvavimo linija (*Lifeline*)

Pranešimas (*Message*)

Sąveikos fragmentas (*Interaction fragment*):

- *occurrence* – pranešimo siuntimo ar gavimo įvykis
- *execution* – aktyvacija, kai objektas vykdo veiksmą
- *state invariant* – būsenos invariantas
- *combined fragment* – fragmentas, turintis valdymo operatorių
- *interaction use* – kitos sąveikų diagramos panaudojimas

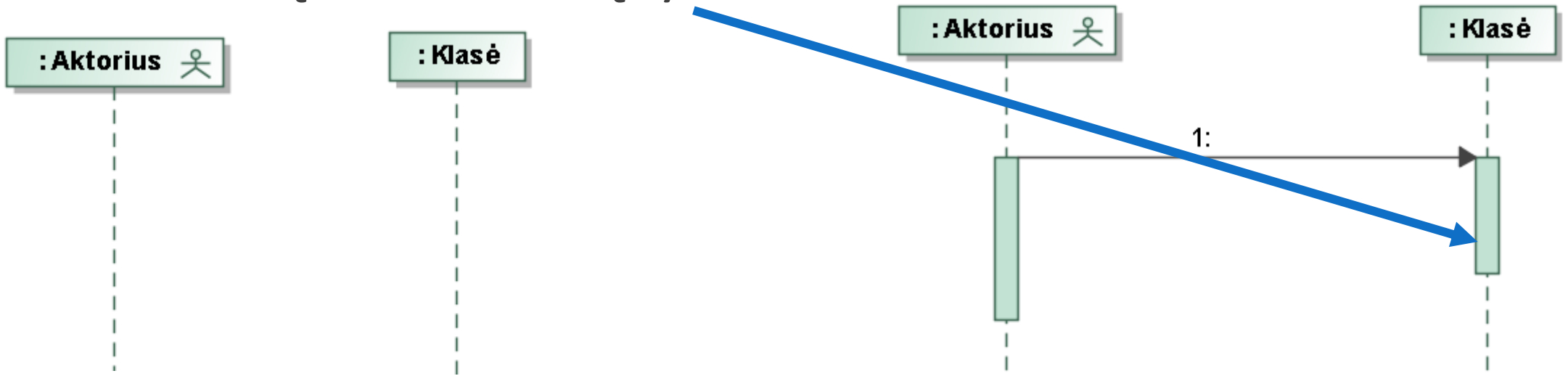
Gyvavimo linija (*Lifeline*) ir aktyvacijos linija (*Execution*)

ktu

Gyvavimo linija vaizduoja sąveikos dalyvį (objektas: Klasė)

Dažniausiai objektai gyvuoja visos sąveikos metu, tačiau gali būti sukuriami ir sunaikinami

Aktyvacijos linija reiškia, kad objektas yra aktyvioje būsenoje – vykdo veiksmą arba laukia įvykio.



Pranešimas (*Message*)

ktu

*The semantics of a complete Message is simply the trace <sendEvent, receiveEvent> **

Pranešimas apibrėžia gyvavimo linijų komunikaciją

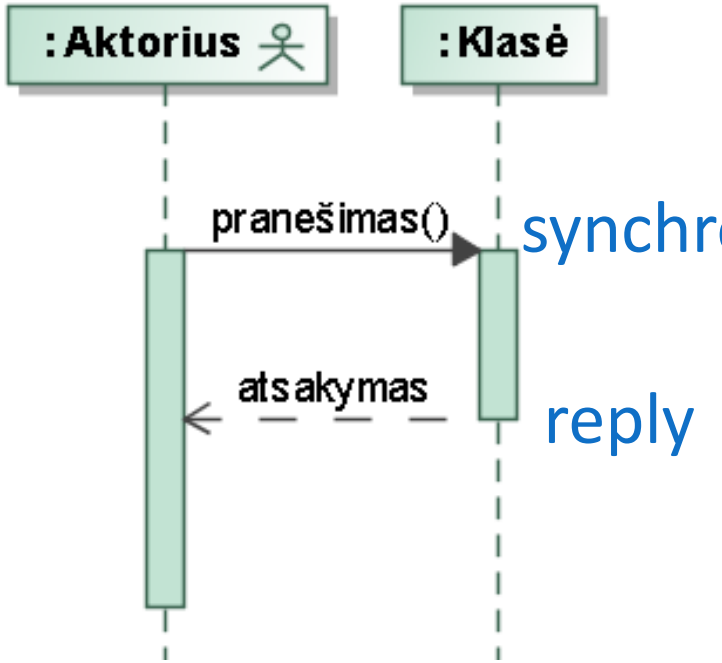
Pranešimas gali būti:

- sinchroninis (kvietėjas laukia atsakymo)
- asinchroninis (kvietėjas nelaukia atsakymo)
- operacijos kvietimas ar atsakymas
- signalo siuntimas
- gyvavimo linijos objekto kūrimas ar naikinimas

*OMG UML 2.5 specification

Pranešimų tipai

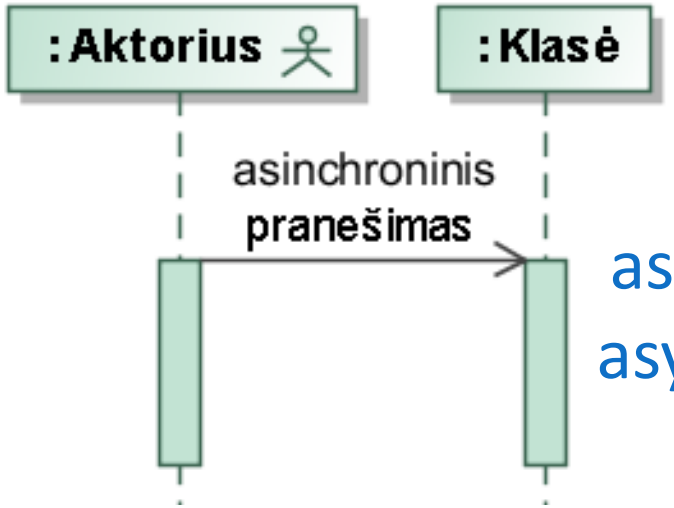
ktu



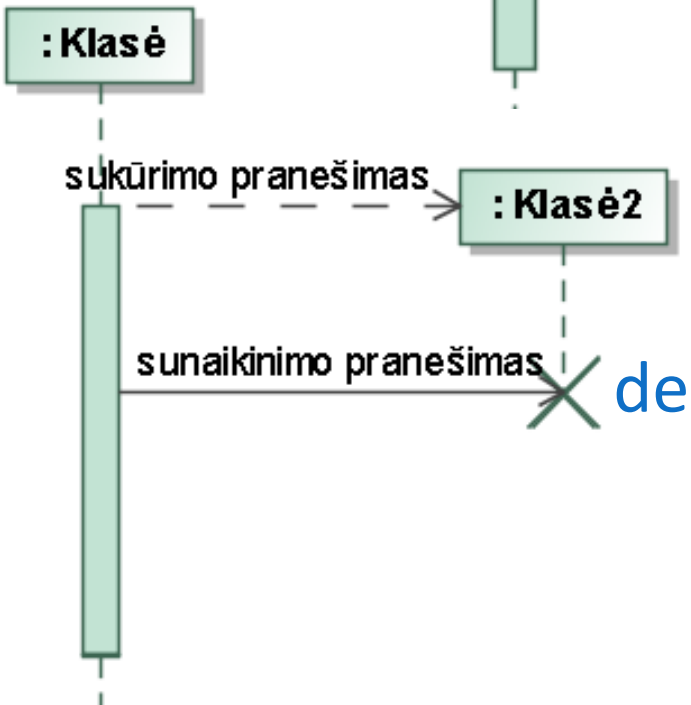
synchronous call

reply

Iškvietusi klasė atsakymo laukia tik po *SynchCall* tipo pranešimo!



asynchronous call / asynchronous signal

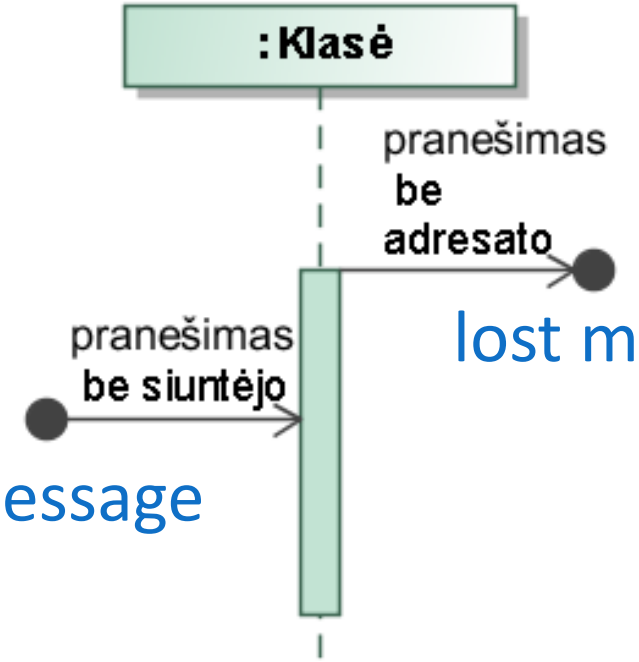


create

delete

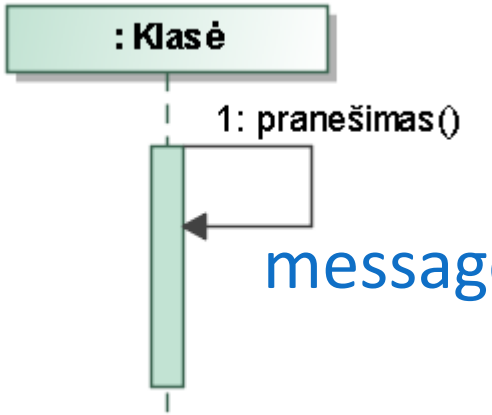
Pranešimų tipai

ktu

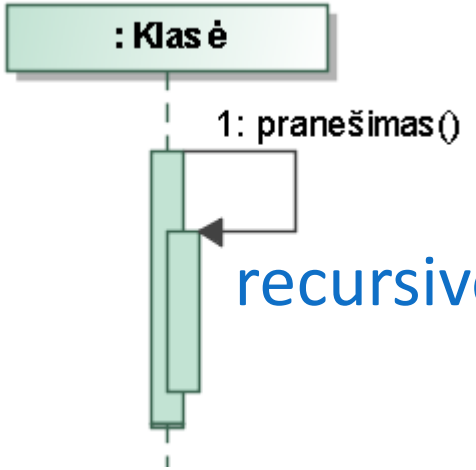


lost message

found message



message to self



recursive message

SynchCall tipo pranešimas

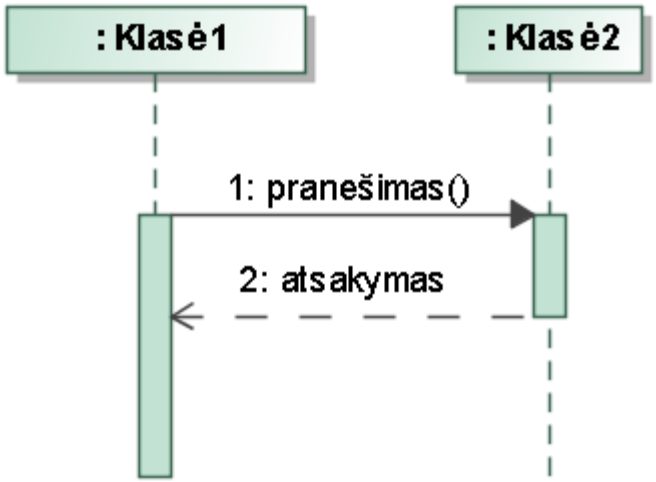
ktu

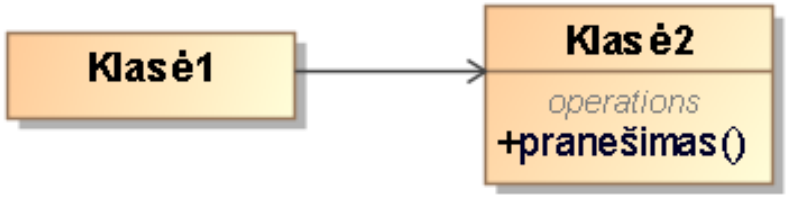
Žymi operacijos kvietimą, po kurio kviečianti gyvavimo linija laukia atsakymo pranešimo (*Reply*).

- laikysimės taisyklės, kad atsakymo pranešimą reikia rodyti visada, kai tik įmanoma, ir būtina nurodyti, kas tame pranešime grąžinama

Jei sekų diagramoje rodome operacijos kvietimo pranešimą:

- klasė (pranešimo gavėja) turi tokią operaciją
- tarp klasių yra ryšys



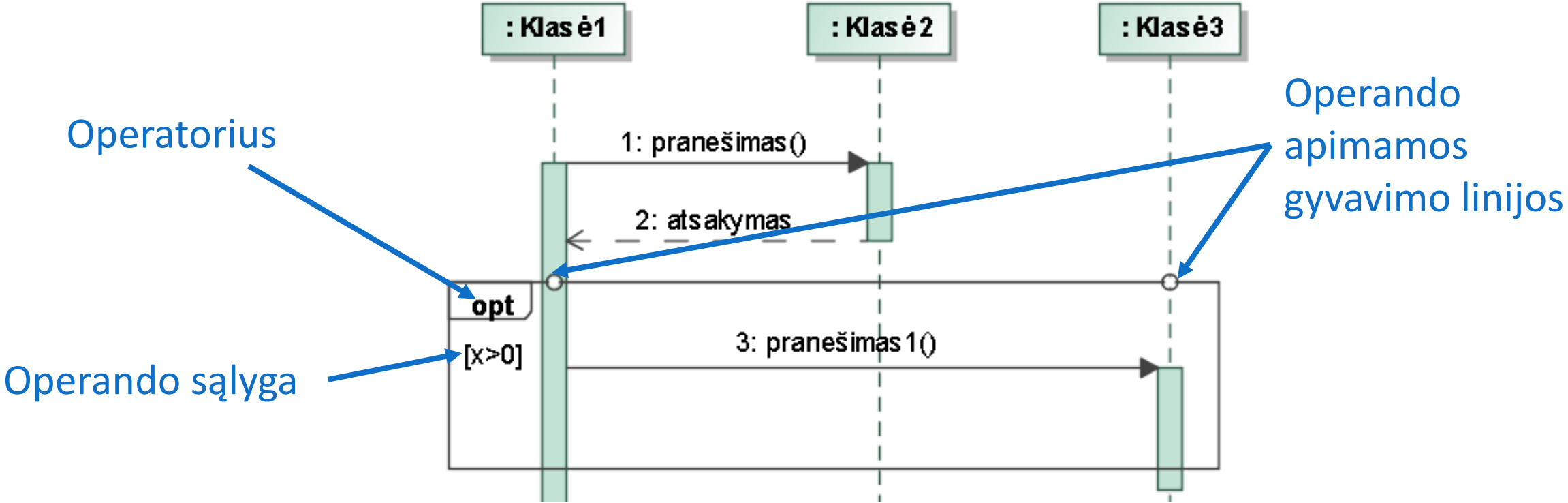


Kombinuotas fragmentas *(Combined fragment)*

ktu

Kiekvienas kombinuotas fragmentas turi valdymo operatorių ir bent vieną operandą su sąlyga (*guard*).

Sekų diagramoje naudojamas kombinuotas fragmentas galioja toms gyvavimo linijoms, kurias jis apima



Combined fragment operatorių tipai

ktu

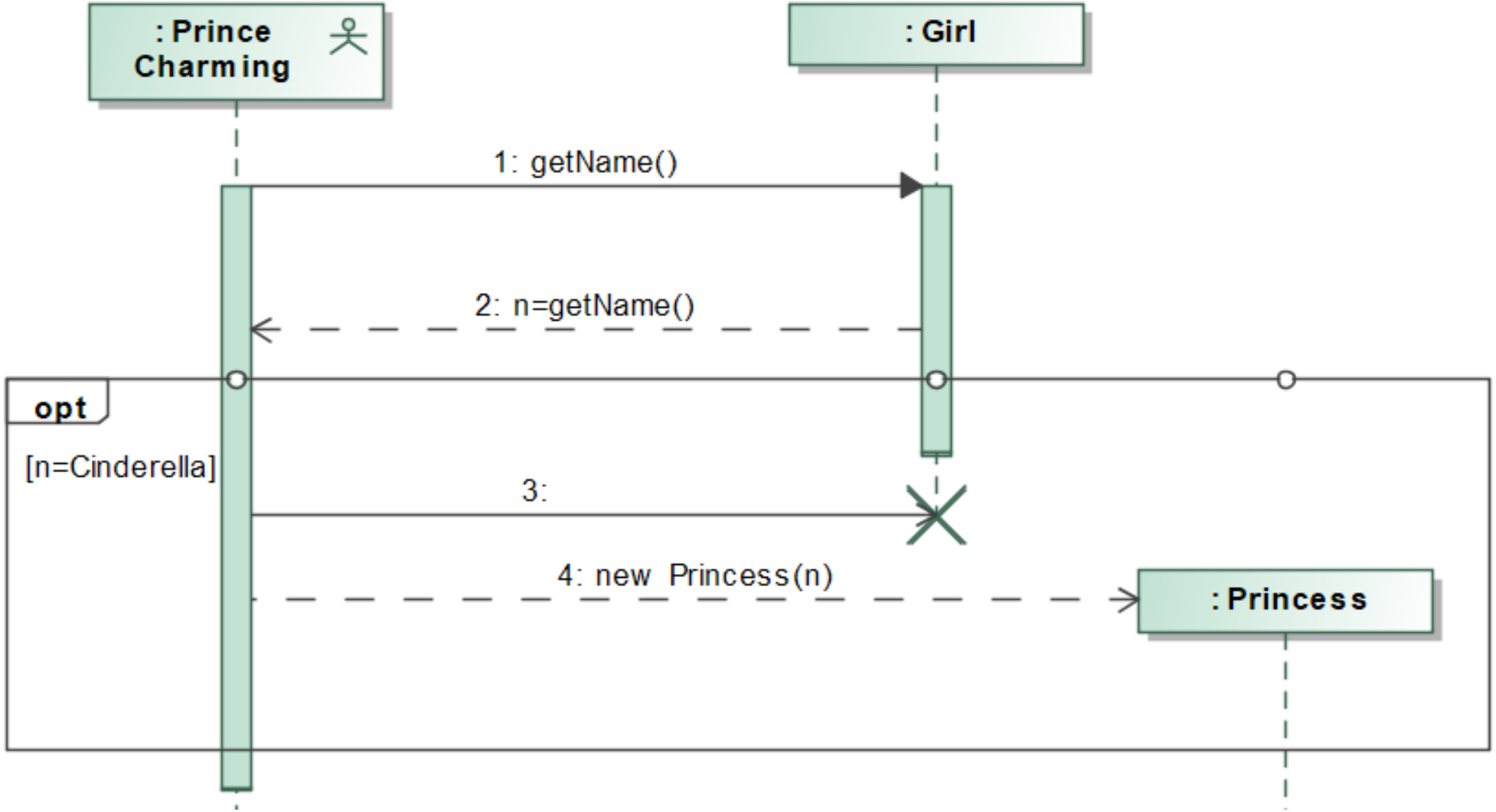
Dažniau naudojami:

- **alt** – alternatyva, vykdomas vienas iš operandų
- **opt** – operandas vykdomas, jei galioja sąlyga
- **loop** – ciklas
- **par** – operandai vykdomi lygiagrečiai

Kiti operatoriai: break, strict, seq, critical, ignore, consider, assert, neg

Pavyzdys

ktu

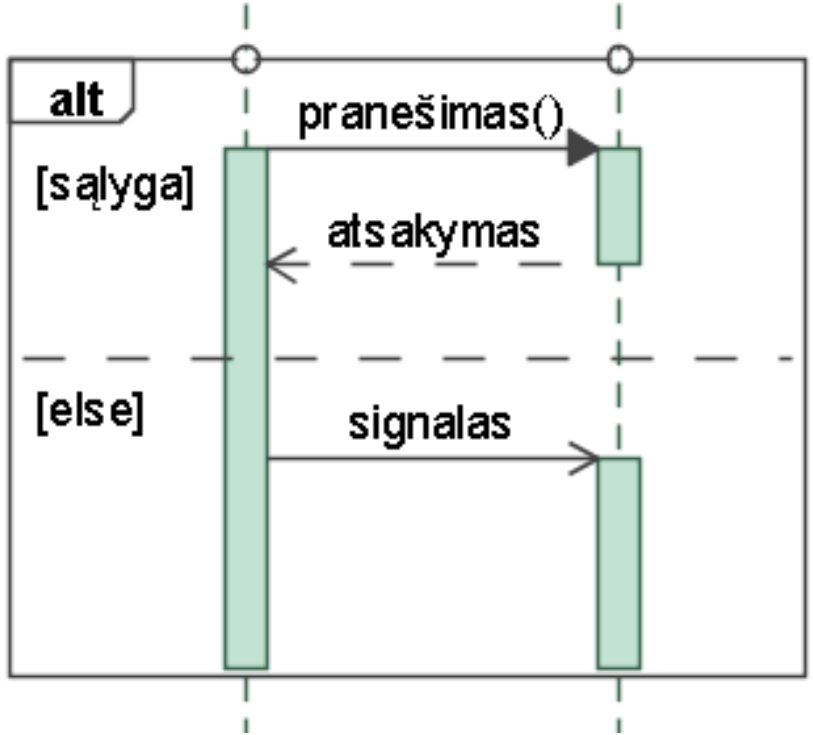


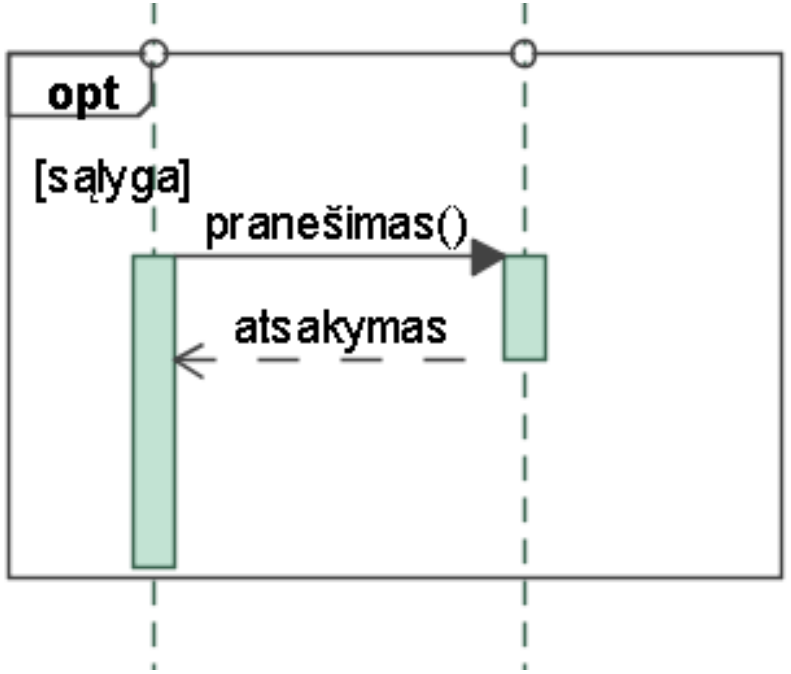
Operatoriai alt ir opt

ktu

alt (*alternative*) operatorius gali turėti ir daugiau nei du operandus

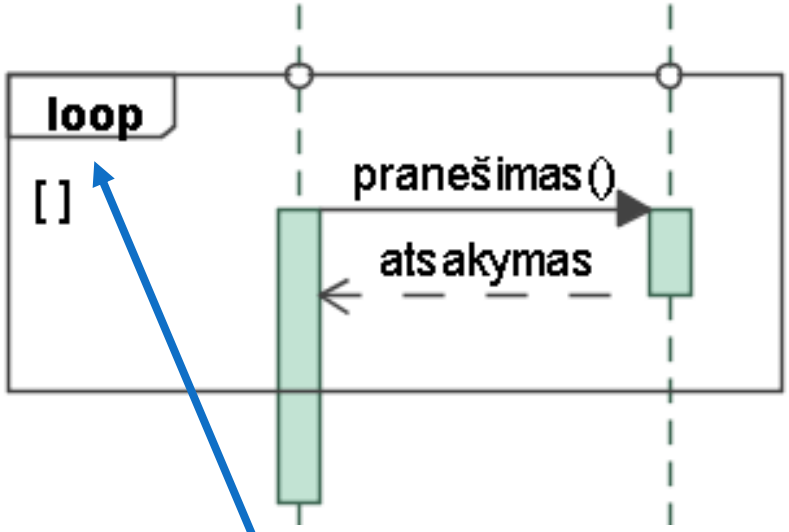
opt (*optional*) visada turi tik vieną operandą



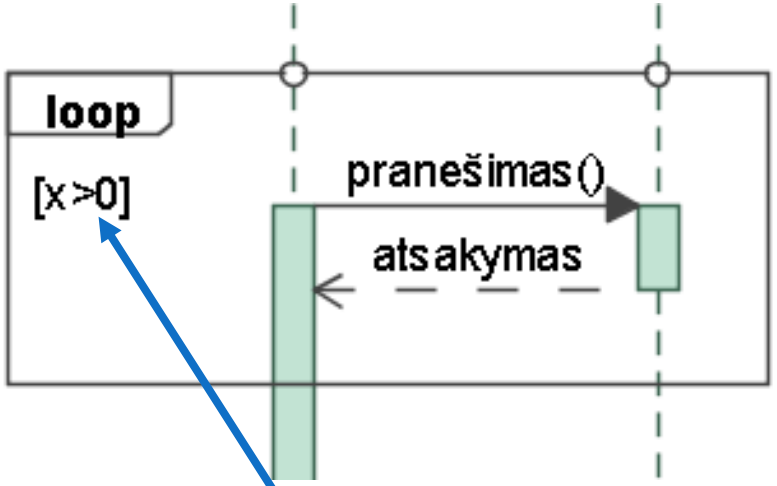


Operatorius loop

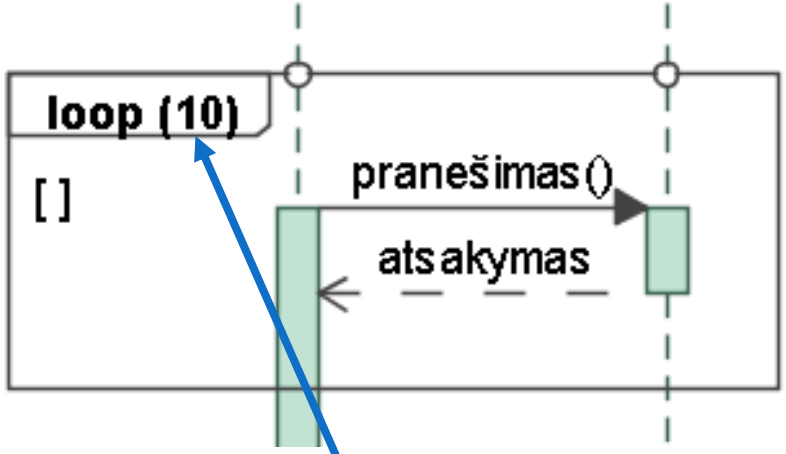
ktu



amžinas ciklas



ciklo sąlyga

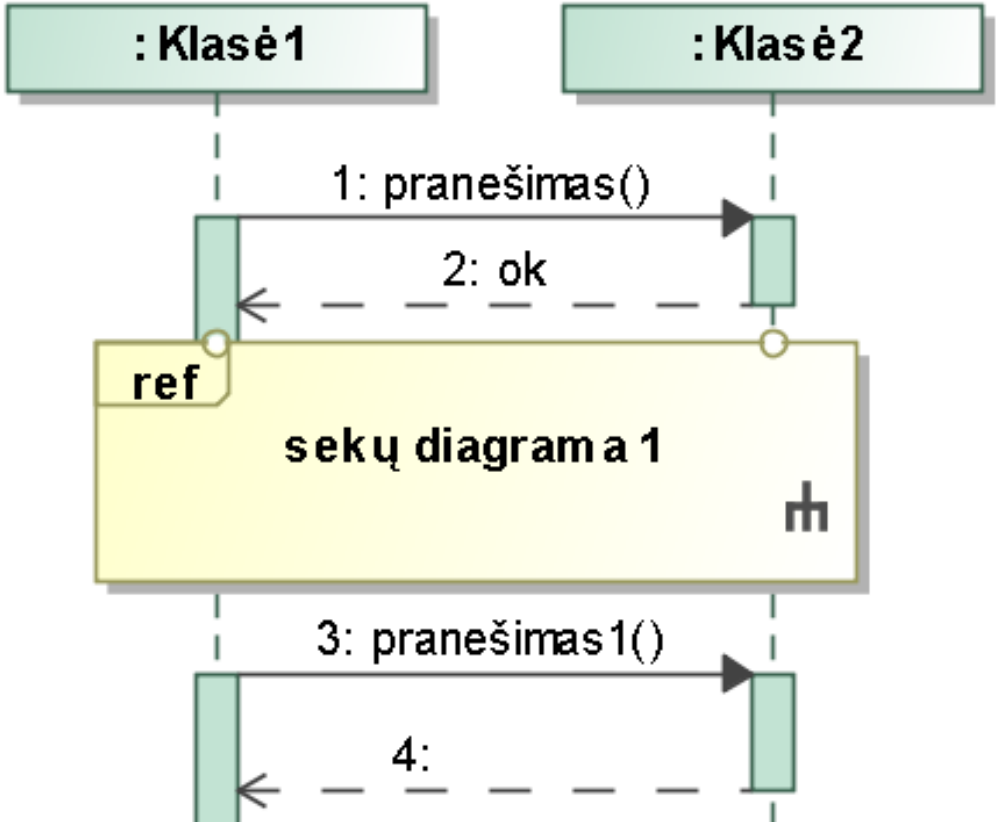


ciklo kartojimų skaičius

Interaction use fragmentas

ktu

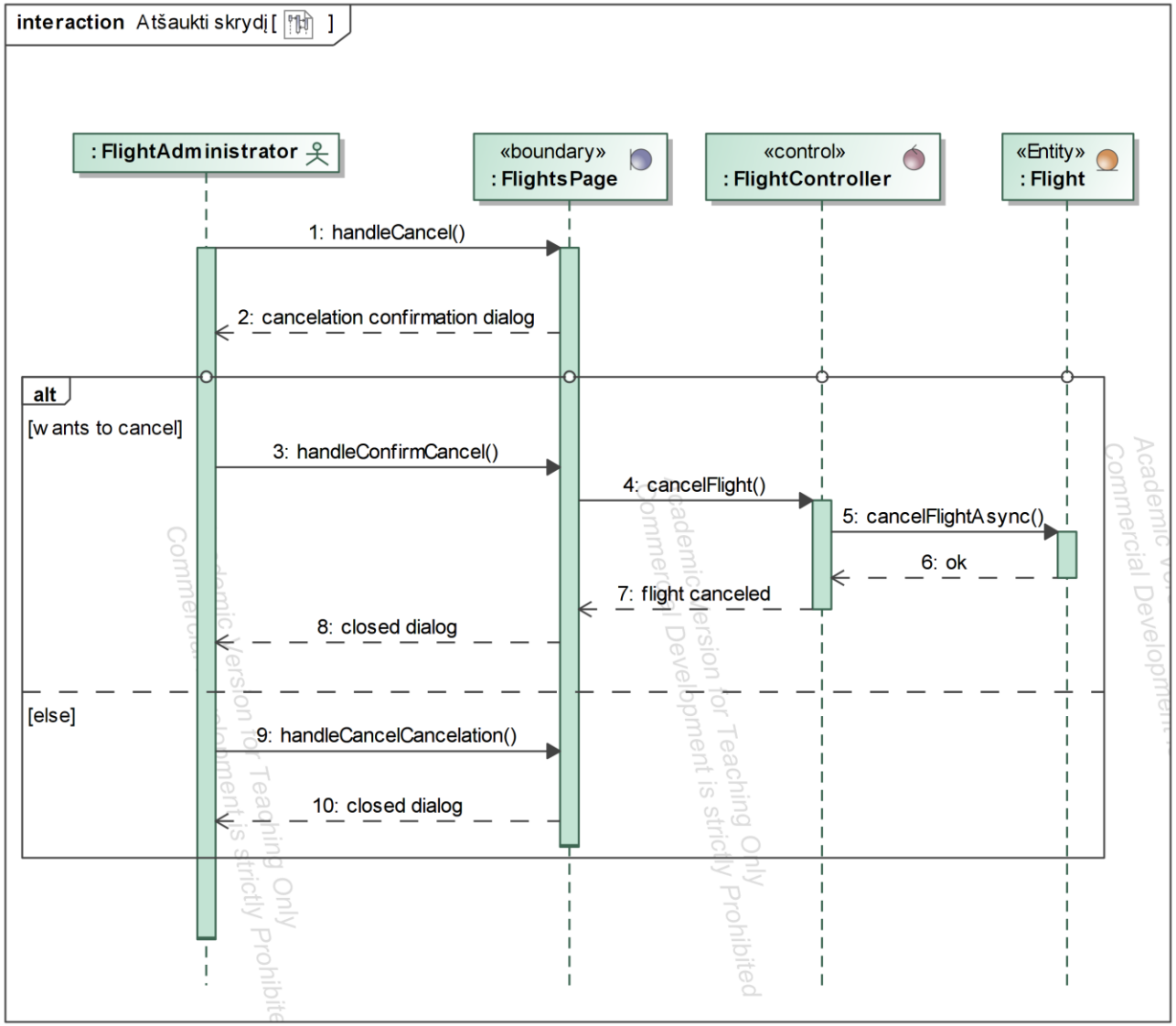
Sekų diagramoje galima naudoti (iškviesti) kitą sąveikų diagramą, tam naudojamas *ref* operatorius

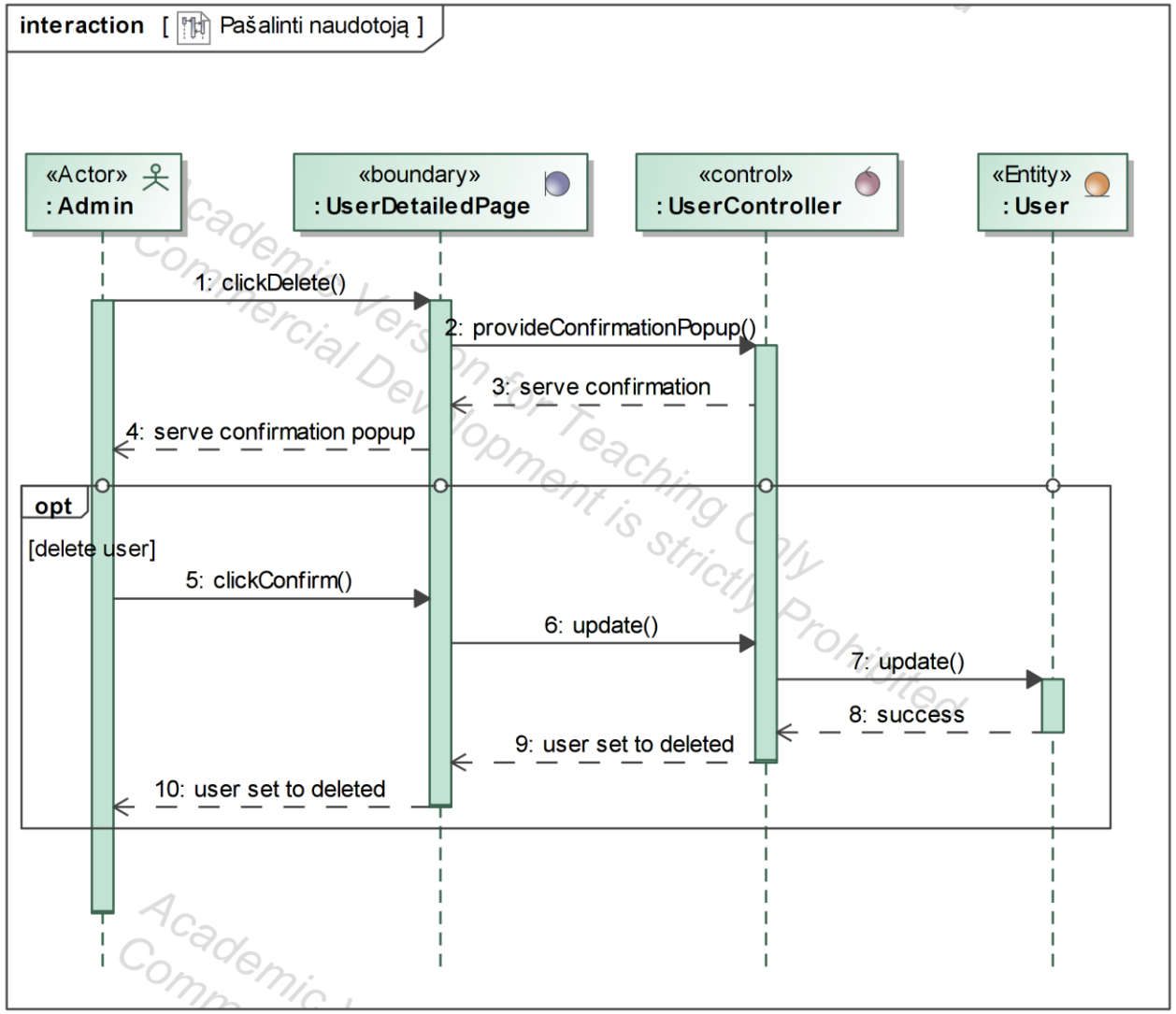


Jei panaudojimo atvejų diagramoje tarp panaudojimo atvejų yra <<include>> arba <<extend>> ryšys, kur sekų diagramose turite panaudoti *ref* operatorių?

Pavyzdžiai

ktu

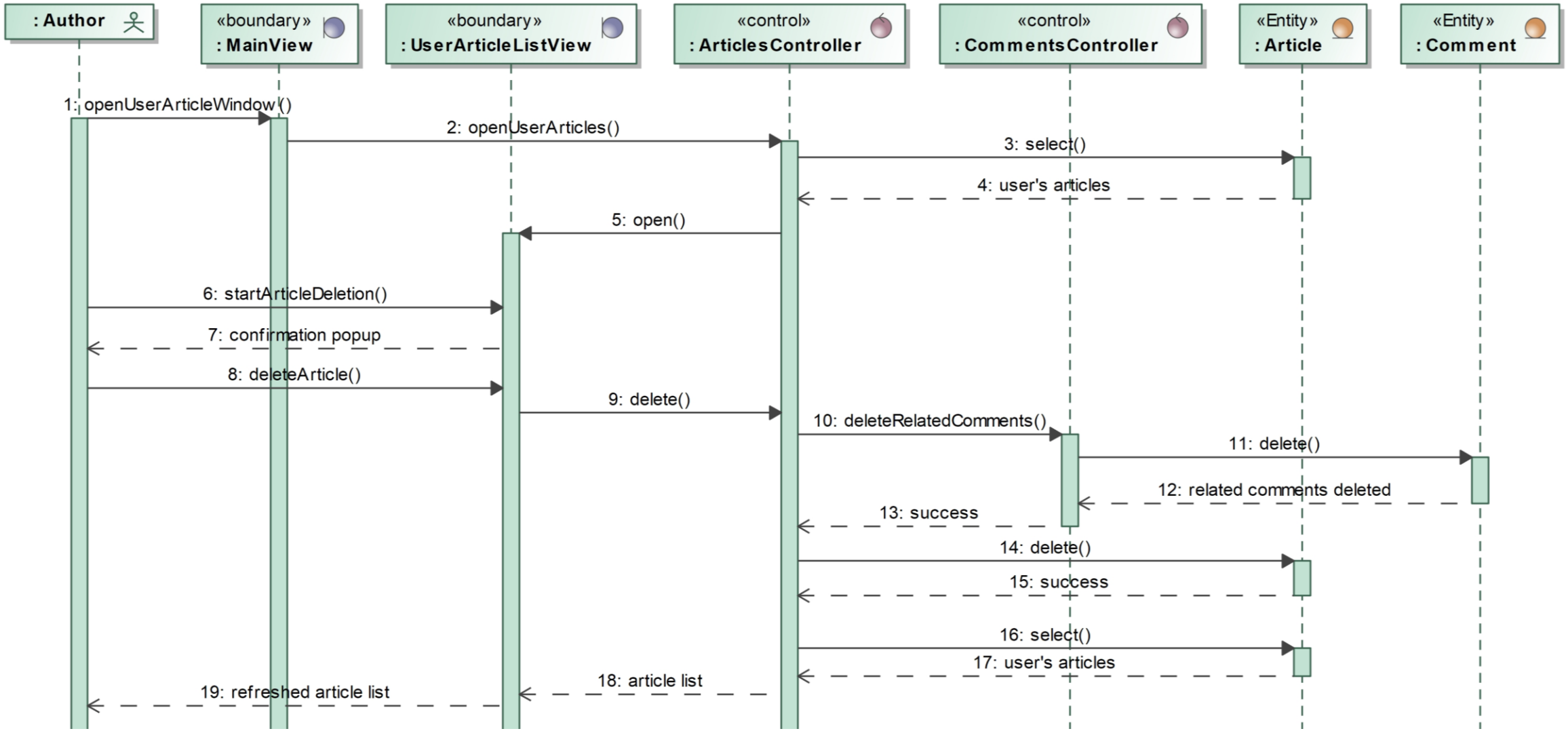




Pavyzdys

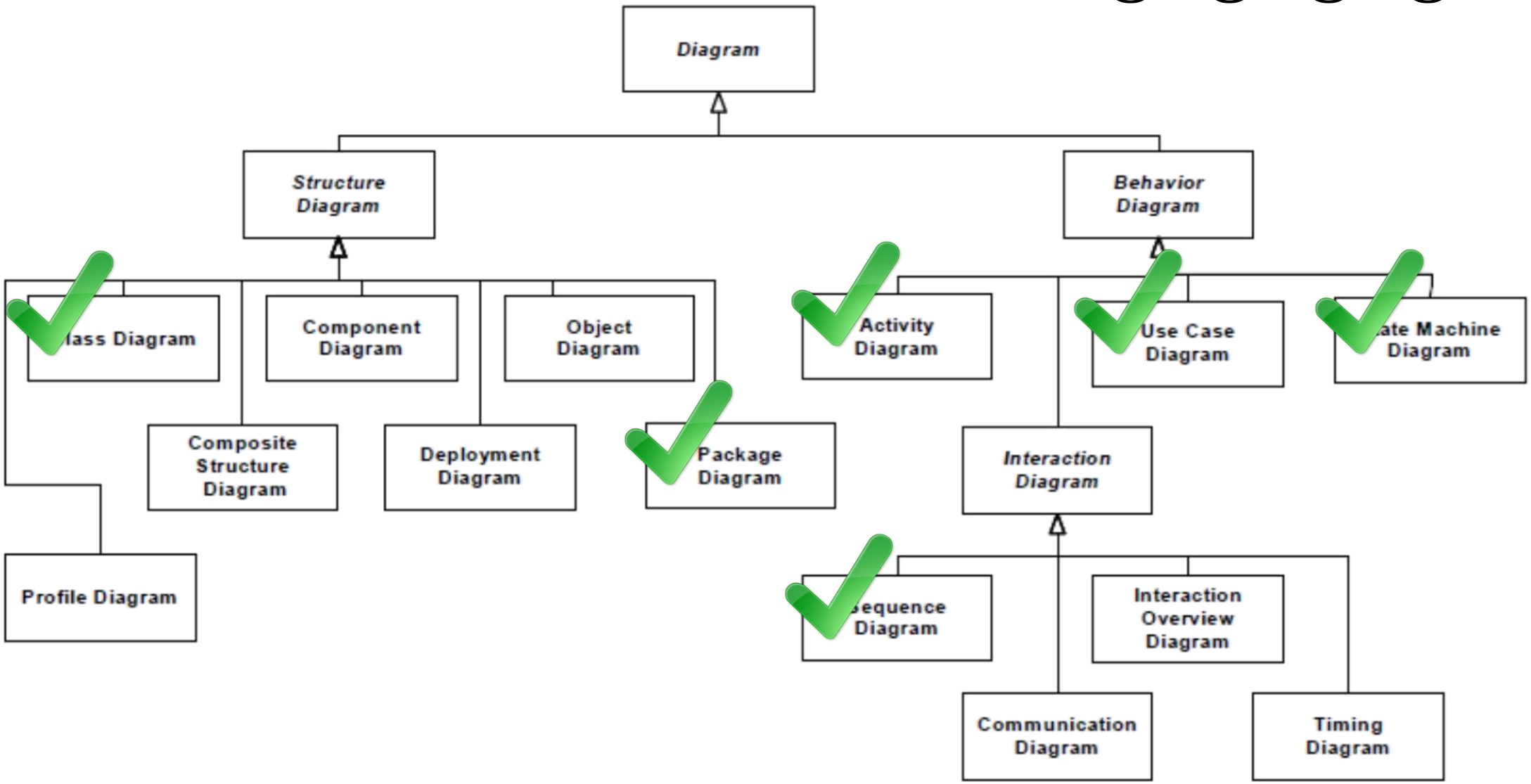
Pašalinti straipsnį

ktu



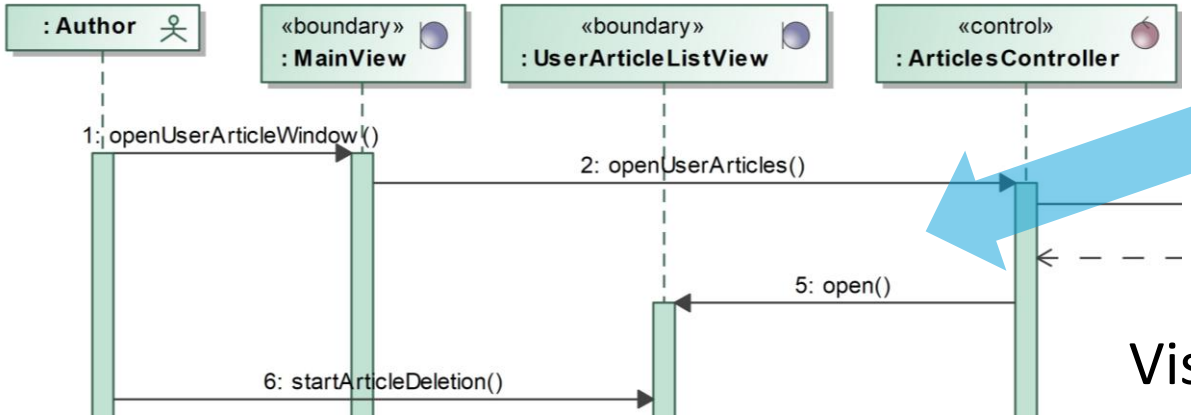
UML 2.5 diagramos

ktu



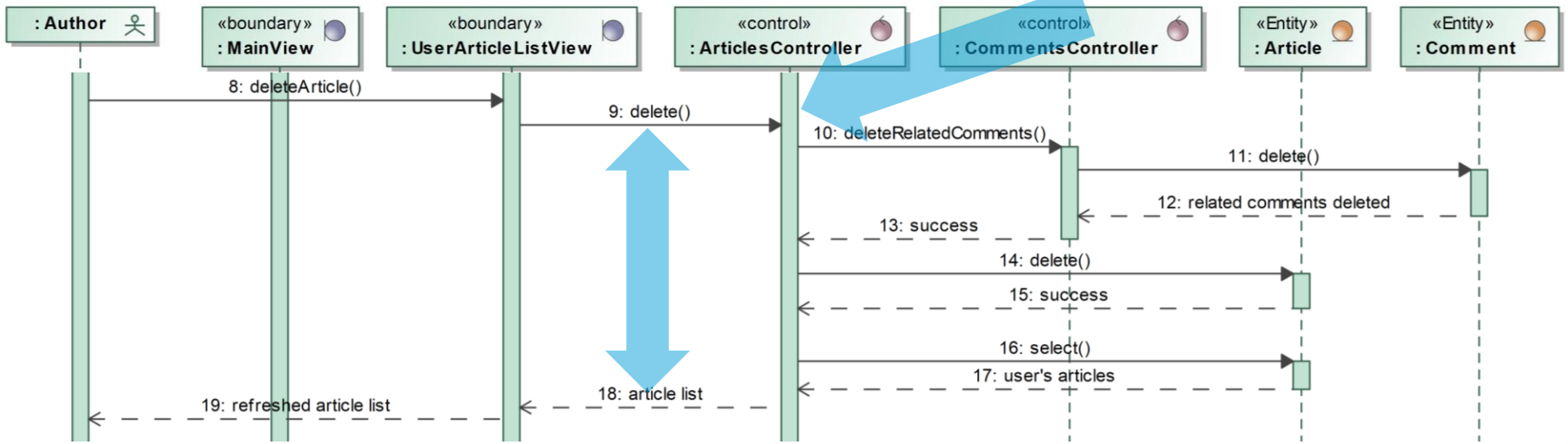
Susitarkim (laboratoriniams darbams)

ktu



Rodydami navigaciją tarp langų, atsakymo pranešimų nerodome

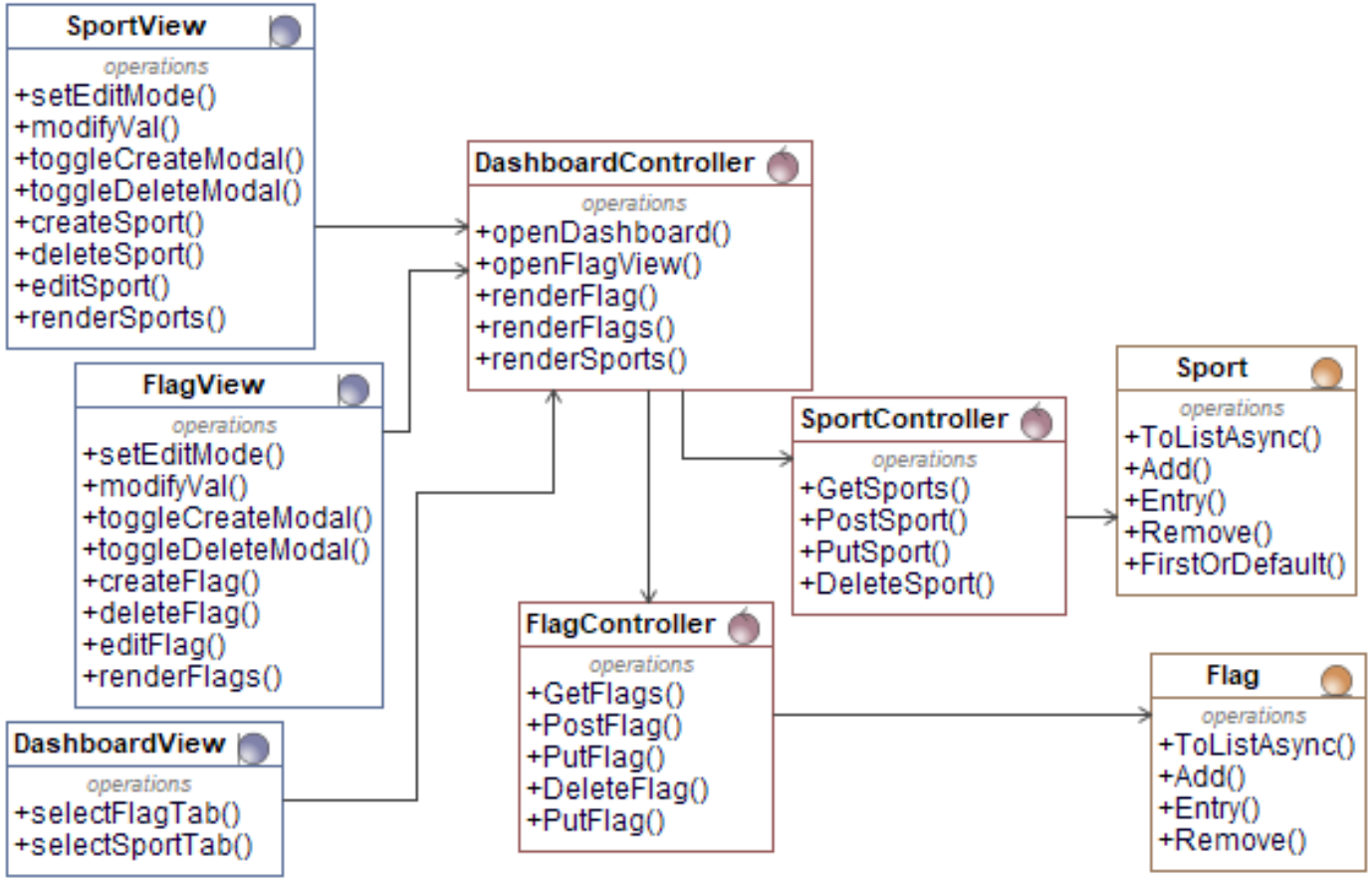
Visais kitais atvejais, atsakymo pranešimus <u>privaloma</u> rodyti kiekvienam *SynchCall*



Projekto klasių diagrama

ktu

Sudarydami sekų diagramas, kuriame operacijas klasėms

Atkreipkite dėmesį:

- ar visos operacijos iš klasių diagramos naudojamos sekų diagramose?
- ar visos sekų diagramose rodomos operacijos matomos klasių diagramoje?